Artificial Intelligence & Machine Learning

Task 2 — Feature Engineering, Model Optimization & Performance Comparison

Submitted by: Abhilasha Singh | Date: June 19, 2026 | Maincrafts Technology

1. Objective

The objective of Task 2 is to go beyond basic model training (covered in Task 1) and learn how Machine Learning engineers improve models in real-world projects. This task focuses on preparing data correctly through feature scaling, training multiple regression algorithms, evaluating them using standard metrics, and selecting the best-performing model with proper justification.

2. Dataset Used

The **California Housing Dataset** was used, which is built into the scikit-learn library. It contains real housing-related data from California census records. The target variable is **Median House Value (HousePrice)**.

Input Features:

Feature	Description
MedInc	Median income of households in the area
HouseAge	Median age of the houses
AveRooms	Average number of rooms per household
AveBedrms	Average number of bedrooms per household
Population	Total population of the block
AveOccup	Average number of occupants per household
Latitude	Geographic latitude of the block
Longitude	Geographic longitude of the block

3. Methodology

Step 1 — Import Libraries

All required Python libraries were imported: pandas and numpy for data handling, matplotlib for visualization, and scikit-learn for machine learning models, preprocessing, and evaluation metrics.

Step 2 — Load the Dataset

The California Housing Dataset was loaded using fetch_california_housing(as_frame=True). The features and target column (HousePrice) were combined into a single DataFrame for inspection.

Step 3 — Separate Features and Target

The input features (X) were separated from the target variable (y = HousePrice). This is required because ML models are trained on X and learn to predict y.

Step 4 — Feature Scaling (Critical Step)

StandardScaler was applied to X. Features in the dataset exist on very different numeric ranges (e.g., Population: 3–35,000 vs AveRooms: 1–20). Without scaling, large-valued features dominate and model performance becomes unstable. After scaling, all features have mean = 0 and standard deviation = 1.

Step 5 — Train-Test Split

The scaled data was split into 80% training and 20% testing using train_test_split with random_state=42. This ensures the model is evaluated on unseen data, not the data it was trained on.

Step 6 — Train Multiple Models

Three regression models were trained: (1) Linear Regression as the baseline, (2) Ridge Regression to reduce overfitting using regularization, and (3) Decision Tree Regressor with max_depth=5 to capture non-linear patterns.

Step 7 — Evaluate and Compare

Each model was evaluated using RMSE (Root Mean Squared Error) and R-squared score. Results were stored in a comparison table for side-by-side analysis.

Step 8 — Visual Validation

A scatter plot of Actual vs Predicted house prices was created for the best model. Points closer to the red diagonal line indicate better predictions.

4. Model Performance Comparison

The table below shows the evaluation metrics for all three models on the test dataset:

Model	RMSE	R^2 Score	Assessment
Linear Regression	0.5674	0.8007	Good baseline
Ridge Regression	0.5674	0.8007	Same as Linear (low multicollinearity)
Decision Tree	**0.5342**	**0.8233**	**Best — lowest error, highest R^2**

*Metric interpretation: **RMSE** — lower values indicate smaller prediction error. **R^2 Score** — higher values (max 1.0) indicate the model explains more variance in the target variable.*

5. Conclusion & Model Selection

The **Decision Tree Regressor** (max_depth=5) was selected as the best-performing model for this task. It achieved an R^2 score of **0.8233**, meaning it explains **82.33%** of the variance in California house prices, and the lowest RMSE of **0.5342**. This outperformed both Linear Regression and Ridge Regression, which scored identically at R^2 = 0.8007. The Decision Tree performed better because it was able to capture **non-linear relationships** between features and house prices that linear models cannot model.

Key Learnings from this Task:

- Feature scaling is critical — it ensures fair learning across all features.
- Multiple models should always be compared — no single algorithm works best for every dataset.
- RMSE and R^2 are complementary metrics — use both together to judge model quality.
- Overfitting can be controlled using regularization (Ridge) or depth limits (Decision Tree).
- Visual validation (Actual vs Predicted plot) confirms what numbers alone may not reveal.

6. Tools & Technologies Used

Tool / Library	Purpose
Python 3	Core programming language
Jupyter Notebook	Interactive coding environment
pandas	Data loading, manipulation, and analysis
NumPy	Numerical computations (RMSE calculation)
scikit-learn	Dataset, preprocessing, models, and metrics
matplotlib	Data visualization (scatter plots, bar charts)
joblib	Saving and loading the trained model (optional)